Chapter No 4 – Number Systems and Logic Gates

C E – 101 : C & P F

Computing and Programming Fundamentals

Batch 2014

Sir Syed University of Engineering & Technology
Computer Engineering Department
University Road, Karachi-75300, PAKISTAN

Compiled By:

Syed Atir Iftikhar

CE - 101 : Computing and Programming Fundamentals

- **Course Objectives:**
  - Upon successful completion of this course, the student will be able to:
    - Fundamentals of Computer Engineering and Information Technology
      - Overview of History, Classification and Components of Computers
      - Number Systems and Logic Gates
      - Overview of Software, Operating Systems, Networks and Internet
    - Introduction about Programming
      - Basic Building Blocks, Loop, Decision Making

2014

CPF Books

- **Text Book:**

  1. **Introduction to Computers** (7th Edition)
     By Peter Norton
  2. **Turbo C Programming For The PC** (Revised Edition)
     By Robert Lafore

- **Reference Books:**

  1. **Computer, Communications and Information.**
     By Sarah Hutchinson and Stacey Sawyer
  2. **Let Us C**
     By Yashavant Kanetkar

Marks Distribution

Mid Term	_____	20
Lab Work + Project	_____	20
Quiz	_____	5
Assignment + Class Performance	_____	5
Semester Final Paper	_____	50
Total Marks	_____	**100**

- ***Performance Bonus*** _____ ***5 Marks***

$\mathcal{CPF}$

Course Instructors

- **Syed Aatir Iftikhar** satirs@hotmail.com

  Lecturer, CED

  Room No: BF-03

  Section B

2014

Course Outline

✓PART A

- History of Computer
- Classification of Computer
- Basic Components
- CPU, I/O, Peripheral Devices, Storage
- Von Neumann Architecture
- Number Systems
- Binary Numbers
- Boolean Logic
- Software
- Operating Systems
- Networks and Internet

✓PART B

- Types of Programming Languages
- Algorithm
- Flow Chart
- Introduction to C Language Programming
- Basic Building Blocks
- Loops
- Decision Making Statements

2014

$\mathcal{CPF}$

CE – 101: Computing and Programming Fundamentals

Chapter 4

Batch 2014

Number Systems And Logic Gates

Compiled By: Syed Atir Iftikhar satirs@hotmail.com

Chapter 4 - Contents

- Know the different types of numbers
- Describe positional notation
- Convert numbers in other bases to base 10
- Convert base 10 numbers into numbers of other bases
- Describe the relationship between bases 2, 8, and 16
- Explain computing and bases that are powers of 2
- Logic Gates and Boolean Algebra

2 0 1 4

10

Numbers

- **Natural Numbers corrent defination**
  - Zero and any number obtained by repeatedly adding one to it.
  - Examples: 100, 0, 45645, 32

- **Negative Numbers**
  - A value less than 0, with a – sign
  - Examples: -24, -1, -45645, -32

Numbers

- **Integers**
  - A natural number, a negative number, zero
  - Examples: 249, 0, - 45645, - 32

- **Rational Numbers**
  - An integer or the quotient of two integers
  - Examples: -249, -1, 0, ¼ , - ½

$\mathcal{CPF}$

Number System

- "A set of values used to represent different quantities is known as Number System".
- The digital computer represents all kinds of data and information in binary numbers. It includes audio, graphics, video, text and numbers. The total number of digits used in a number system is called its base or radix. The base is written after the number as subscript such as 15_{10}.
- Some important number systems are as follows.
  - Decimal number system
  - Binary number system
  - Octal number system
  - Hexadecimal number system

Decimal Numbering systems

- Base: 10
- Digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Representation 5234

Thousands	Hundreds	Tens	Units
5	2	3	4

- Example: 5234_{10}

$10^3 = 1000 \quad 10^2 = 100 \quad 10^1 = 10 \quad 10^0 = 1$

5 2 3 4

$5{,}234 = 5 \times 1000 + 2 \times 100 + 3 \times 10 + 4 \times 1$

2014

7

$\mathcal{CPF}$

Positional Notation

Continuing with our example...
642 in base 10 *positional notation* is:

$$6 \times 10^2 = 6 \times 100 = 600$$
$$+ 4 \times 10^1 = 4 \times 10 = 40$$
$$+ 2 \times 10^0 = 2 \times 1 = 2 \quad = 642 \text{ in base 10}$$

This number is in base 10

The power indicates the position of the number

Positional Notation

As a formula:

$$d_n * R^{n-1} + d_{n-1} * R^{n-2} + \ldots + d_2 * R_1 + d_1$$

n is the number of digits in the number

d is the digit in the i^{th} position in the number

R is the base of the number

642 is: $6_3 * 10^2 + 4_2 * 10^+ 2_1$

Positional Notation

What if 642 has the base of 13?

$+ 6 \times 13^2 = 6 \times 169 = 1014$

$+ 4 \times 13^1 = 4 \times 13 = 52$

$+ 2 \times 13^0 = 2 \times 1 = 2$

$= 1068$ in base 10

642 in base 13 is equivalent to 1068 in base 10

Binary Numbering System

- Base: 2
- Digits: 0, 1
- binary number: 110101_2

positional powers of 2:	2^5	2^4	2^3	2^2	2^1	2^0
decimal positional value:	32	16	8	4	2	1
binary number:	1	1	0	1	0	1

$\mathcal{CPF}$

Converting Binary to Decimal

- What is the decimal equivalent of the binary number 1101110 ?

$$
\begin{aligned}
1 \times 2^6 &= 1 \times 64 &&= 64 \\
+\,1 \times 2^5 &= 1 \times 32 &&= 32 \\
+\,0 \times 2^4 &= 0 \times 16 &&= 0 \\
+\,1 \times 2^3 &= 1 \times 8 &&= 8 \\
+\,1 \times 2^2 &= 1 \times 4 &&= 4 \\
+\,1 \times 2^1 &= 1 \times 2 &&= 2 \\
+\,0 \times 2^0 &= 0 \times 1 &&= 0 \\
& & &= 110 \text{ in base } 10
\end{aligned}
$$

2 0 1 4

19

CPF - Chapter No 4 : Number Systems and Logic Gates

Binary to Decimal Conversion

- To convert to base 10, add all the values where a one digit occurs.

Ex: 110101_2

positional powers of 2:	2^5	2^4	2^3	2^2	2^1	2^0
decimal positional value:	32	16	8	4	2	1
binary number:	1	1	0	1	0	1

$$32 + 16 + 4 + 1 = 53_{10}$$

2 0 1 4

20

CPF - Chapter No 4 : Number Systems and Logic Gates

$\mathcal{CPF}$

Decimal to Binary Conversion

- ***The Division Method.*** Divide by 2 until you reach zero, and then collect the remainders in reverse.

Example 1: $56_{10} = 111000_2$

2) 56	Rem:
2) 28	0
2) 14	0
2) 7	0
2) 3	1
2) 1	1
0	1

Decimal to Binary Conversion

- ***The Subtraction Method:***

Subtract out largest power of 2 possible (without going below zero) each time until you reach 0. Place a one in each position where you were able to subtract the value, and a 0 in each position that you could not subtract out the value without going below zero.

C P F

Decimal to Binary Conversion

- ***The Subtraction Method:***

Example: 56_{10}

56	2^6	2^5	2^4	2^3	2^2	2^1	2^0
- **32**	**64**	**32**	**16**	**8**	**4**	**2**	**1**
24		1	1	1	0	0	0
- **16**							
8							
- **8**							
0							

Answer: $56_{10} = 111000_2$

Octal Numbering System

- Base: 8
- Digits: 0, 1, 2, 3, 4, 5, 6, 7
- Octal number: 1246_8

powers of :	8^4	8^3	8^2	8^1	8^0
decimal value:	4096	512	64	8	1

Octal to Decimal Conversion

- To convert to base 10, beginning with the rightmost digit multiply each **n**th digit by $8^{(n-1)}$, and add all of the results together.

Example: 1246_8

positional powers of 8:	8^3	8^2	8^1	8^0
decimal positional value:	512	64	8	1
Octal number:	1	2	4	6

$$\mathbf{1246_8 = 512 + 128 + 32 + 6 = 678_{10}}$$

Decimal to Octal Conversion

- *The Division Method.* Divide by 8 until you reach zero, and then collect the remainders in reverse.

Example: $\mathbf{4330_{10} = 10352_8}$

	Rem:
8) 4330	
8) 541	2
8) 67	5
8) 8	3
8) 1	0
0	1

$\mathcal{CPF}$

Hexadecimal Numbering System

- Base: 16
- Digits: 0, 1, 2, 3, 4, 5, 6, 7,8,9,A,B,C,D,E,F
- Hexadecimal number: $1F4_{16}$

powers of :	16^4	16^3	16^2	16^1	16^0
decimal value:	65536	4096	256	16	1

Hexadecimal Numbering System

4-Bit Group	Decimal Digit	HexaDecimal Digit
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

$\mathcal{CPF}$

Hexa-Decimal to Decimal Conversion

- To convert to base 10, beginning with the rightmost digit multiply each **n**th digit by $16^{(n-1)}$, and add all of the results together.

Example: $1F4_{16}$

positional powers of 16:	16^3	16^2	16^1	16^0
decimal positional value:	4096	256	16	1
Hexadecimal number:	1	F	4	

$$\mathbf{1F4_{16} = 256 + 240 + 4 = 500_{10}}$$

29

CPF - Chapter No 4 : Number Systems and Logic Gates

Hexa-Decimal to Decimal Conversion

- What is the decimal equivalent of the hexadecimal number DEF?

$$\begin{aligned} & D \times 16^2 = 13 \times 256 = 3328 \\ + \; & E \times 16^1 = 14 \times 16 = 224 \\ + \; & F \times 16^0 = 15 \times 1 = 15 \\ & = 3567 \text{ in base } 10 \end{aligned}$$

30

CPF - Chapter No 4 : Number Systems and Logic Gates

$\mathcal{CPF}$

Decimal to Hexa Conversion

- *The Division Method.* Divide by 16 until you reach zero, and then collect the remainders in reverse.

Example 1: $\mathbf{126_{10}} = \mathbf{7E_{16}}$

16) 126 Rem:

16) 7 14 = E

0 7

Decimal to Hexa Conversion

Example: $\mathbf{810_{10}}$

16^3	16^2	16^1	16^0
4096	**256**	**16**	**1**
	3	2	A

Answer: $810_{10} = \mathbf{32A_{16}}$

Binary to Octal Conversion

- Since the maximum value represented in 3 bit is equal to:

$$2^3 - 1 = 7$$

i.e. using 3 bits we can represent values from 0 –7

which are the digits of the Octal numbering system.

Thus, three binary digits can be converted to one octal digit.

Binary to Octal Conversion

Three-bit Group	Decimal Digit	Octal Digit
000	0	0
001	1	1
010	2	2
011	3	3
100	4	4
101	5	5
110	6	6
111	7	7

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Octal to Binary Conversion

- Example : Convert 742_8 = $\_\_\_\__2$

$7 = 111$

$4 = 100$

$2 = 010$

$$742_8 = 111100010_2$$

CPF - Chapter No 4 : Number Systems and Logic Gates 35

Binary to Octal Conversion

Example : Convert 10100110_2 = $\_\_\_\__8$

$110 = 6$

$100 = 4$

$010 = 2$ (pad empty digits with 0)

$$10100110_2 = 246_8$$

CPF - Chapter No 4 : Number Systems and Logic Gates 36

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Binary to Hexa Conversion

- Since the maximum value represented in 4 bit is equal to:

$$2^4 - 1 = 15$$

i.e. using 4 bits we can represent values from 0 –15 which are the digits of the Hexadecimal numbering system.

Thus, Four binary digits can be converted to one Hexadecimal digit.

Hexa to Binary Conversion

- Example : Convert $3D9_8 =$ $_2$

$3 = 0011$

$D = 1101$

$9 = 1001$

$\mathbf{3D9_8 = 0011\ 1101\ 1001_2}$

Binary to Hexa Conversion

Example: Convert 10100110_2 = $\_\_\_\__8$

0110 = 6

1010 = A

$1010\ 0110_2 = A\ 6_{16}$

Octal to Hexa Conversion

- To convert between Octal to Hexadecimal numbering systems and visa versa convert from one system to binary first then convert from binary to the new numbering system

$\mathcal{CPF}$

Hexa to Octal Conversion

Example : Convert $E8A_{16}$ = $\_\_\_\__8$

$1110\ \ 1000\ \ 1010_2$

111 010 001 010 (group by 3 bits)

$E\ 8\ A_{16} = 7\ 2\ 1\ 2_8$

Octal to Hexa Conversion

Example : Convert 752_8 = $\_\_\_\__{16}$

111 101 010_2 (group by 4 bits)

0001 1110 1010

1 E A

$7\ 5\ 2_8 = 1\ E\ A_{16}$

C P F

Arithmetic in Binary

- Remember: there are only 2 digits in binary: 0 and 1
- Position is key, carry values are used:

$$\begin{array}{r} 10110 \\ +\ 10011 \\ \hline 101001_2 \end{array}$$

2 0 1 4

Carry Values

1 11111
1010111
+1001011
10100010

Binary Subtraction

Remember borrowing?

Apply that concept here:

1 2
2 0 2
1010111
- 111011
0011100

$$\begin{array}{r} 10110 \\ -\ 10011 \\ \hline 00011_2 \end{array}$$

2 0 1 4

$\mathcal{CPF}$

Number System (Summary)

- Binary 0,1
- Octal 0,1,2,3,4,5,6,7
- Decimal 0,1,2,3,4,5,6,7,8,9
- Hexadecimal 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

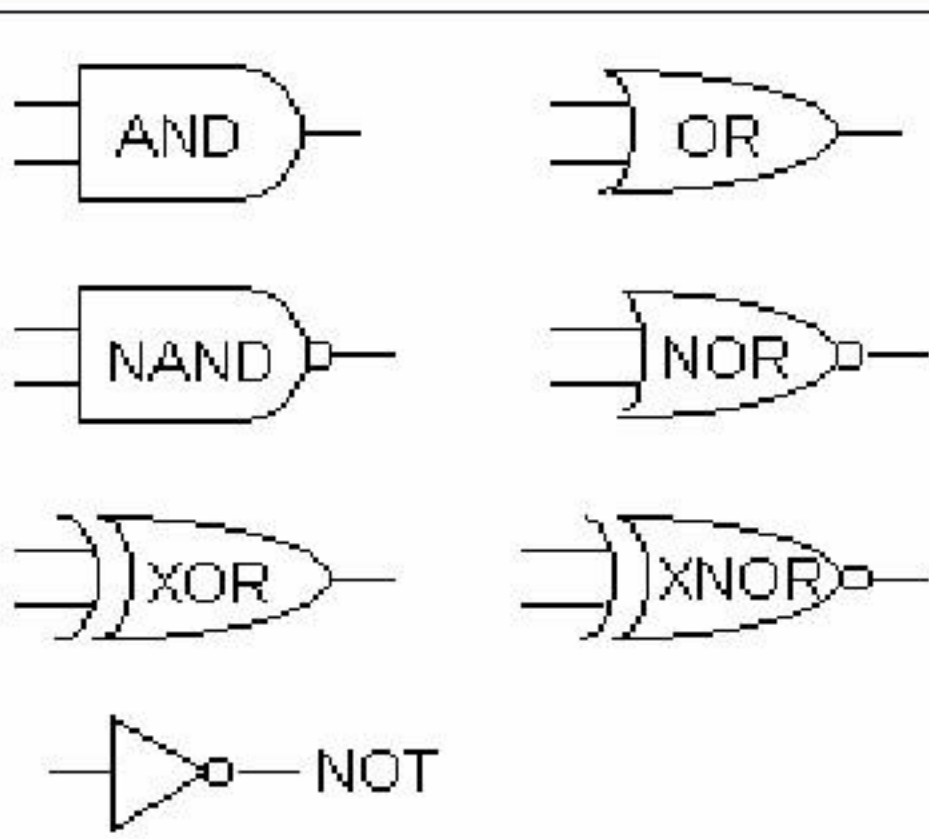
Basic Logic Functions

A	B	A and B	A or B	A nand B	A nor B	A xor B	A xnor B	not A
1	1	1	1	0	0	0	1	0
1	0	0	1	1	0	1	0	0
0	1	0	1	1	0	1	0	1
0	0	0	0	1	1	0	1	1

$\mathcal{CPF}$

Logic Gate Symbols

- AND
- OR
- NAND
- NOR
- XOR
- XNOR
- NOT



2 0 1 4

47

CPF - Chapter No 4 : Number Systems and Logic Gates

Logic Gates ⇉ Logic Gate →

- Logic Gates are the basic building blocks for building electronic (digital) circuits
- They have output terminal, and (n) input terminal(s)
- Output will be 1 or 0, depending on the digital levels of the input terminal(s)
- These gates form the basic building blocks of digital systems; that evaluate digital input levels and produce specific output response
- **7 Logic Gates:**
  - AND, OR, NOT, NAND, NOR, XOR, X-NOR

2 0 1 4

48

CPF - Chapter No 4 : Number Systems and Logic Gates

Logic Gate Notations

- Not A is written $\bar{A}$
- A and B is written $\overline{AB}$
- A or B is written $A + B$
- A xor B is written $A \oplus B$

Related Terminologies

1. Boolean Expression

- An expression which evaluates to either true or false is called a **Boolean Expression**.
- An expression that results in a value of either TRUE or FALSE. For example, the expression $2 < 5$ (2 is less than 5) is a Boolean expression because the result is TRUE.
- All expressions that contain relational operators , such as the less than sign (<), are Boolean.
- The operators -- AND, OR, XOR, NOR, and NOT -- are Boolean operators.
- Boolean expressions are also called comparison expressions, conditional expressions, and relational expressions.

$\mathcal{CPF}$

Related Terminologies

2. Truth Table

- A ***truth table*** shows how a logic circuit's output responds to various combinations of the inputs, using logic 1 for true and logic 0 for false.
- A truth table is a breakdown of a logic function by listing all possible values the function can attain
- All permutations of the inputs are listed on the left, and the output of the circuit is listed on the right. The desired output can be achieved by a combination of logic gates.
- A truth table can be extended to any number of inputs. The input columns are usually constructed in the order of binary counting with a number of bits equal to the number of inputs.

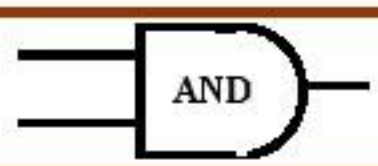
Related Terminologies

3. Logic Circuit

- *Logic Circuit* is a graphical representation of a program using formal logic

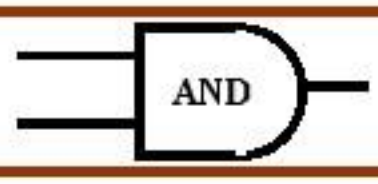
$\mathcal{CPF}$

AND Gate

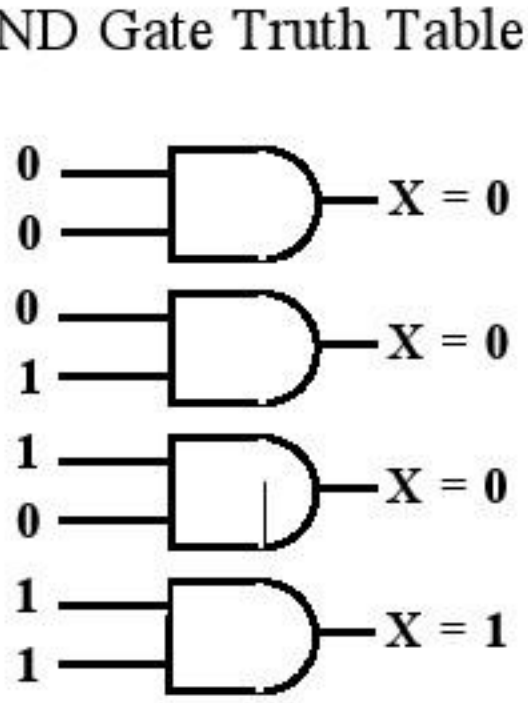


- This is a 2-Input AND Gate…(label inputs & output)
- By convention, letters A, B are used as inputs, and letter(s) X is used as output
- AND Gate Operation is defined as:
  - The output, X, is HIGH if input A *and* input B are both HIGH.
  - **Output is 1 only when all inputs are 1**
- Let's complete a *truth table* for the AND gate

AND Gate



- AND Gate Truth Table

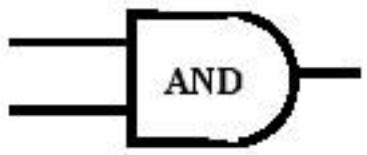


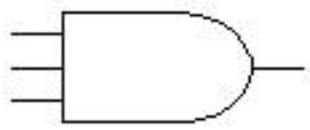
AND Truth Table

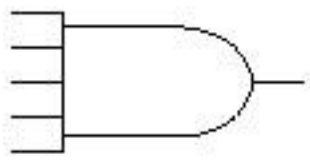
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

AND Gate

- Notation of AND Gate
  - A **AND** B
  - A . B
  - A B



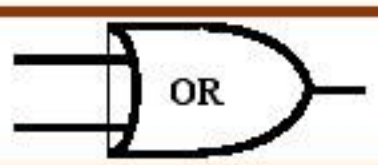




- AND gates may havemore than two inputs
- How many combinations to be listed in a truth table?

2 0 1 4

OR Gate

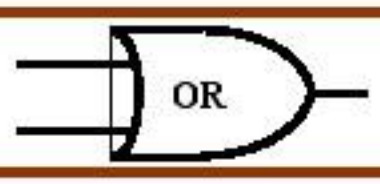


- This is a two-input OR gate…(label inputs & output)
- OR Gate Operation is defined as:
  - *The output at X will be HIGH whenever input A or input B is HIGH or both are HIGH.*
  - **Output is 1 only when any of the inputs is 1**
- Let's complete a *truth table* for the OR gate

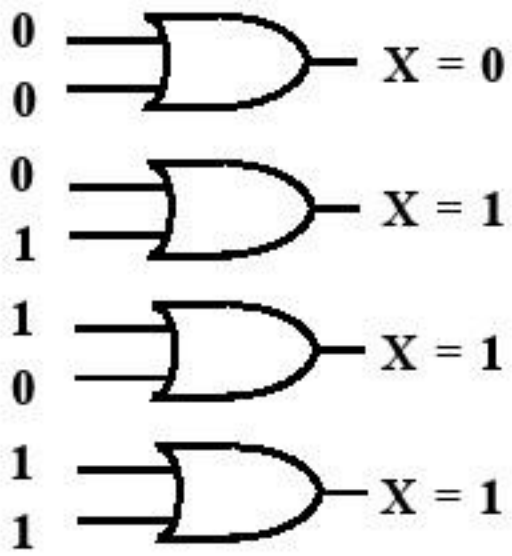
2 0 1 4

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

OR Gate



- OR Gate Truth Table



OR Truth Table

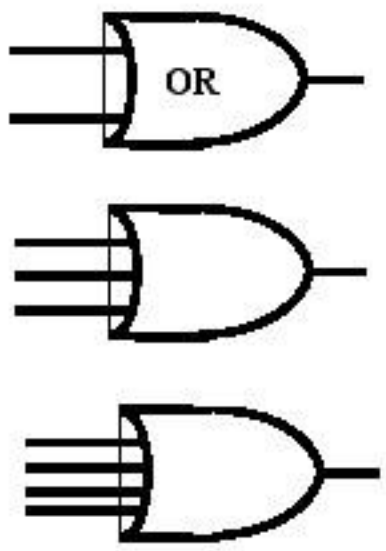
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

2 0 1 4

57

CPF - Chapter No 4 : Number Systems and Logic Gates

- Notation of OR Gate
  - A **OR** B
  - A + B



2 0 1 4

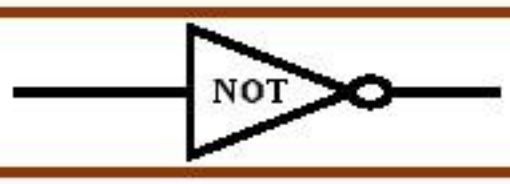
- AND gates may have more than two inputs
- How many combinations to be listed in a truth table?

58

CPF - Chapter No 4 : Number Systems and Logic Gates

$\mathcal{CPF}$

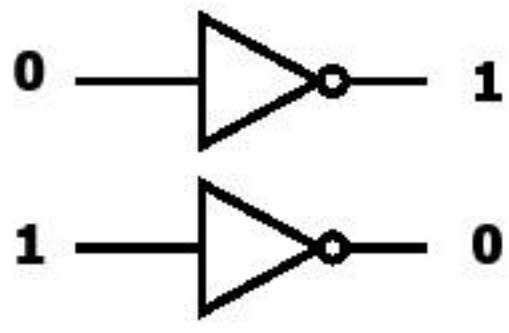
NOT Gate



- Not gate is also known as **Inverter Gate**
- ***A NOT gate is a one-input-one-output logic gate***
- **Notation of NOT Gate: NOT A = $\overline{A}$**

2014

NOT Gate



NOT Truth Table

A	X
0	1
1	0

When A = 0, Output X is NOT 0, Hence = 1

When A = 1, Output X is NOT 1, Hence = 0

2014

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

XOR Gate

- XOR is also called as Exclusive OR
- Operation of XOR gate
  - True if either true but not both
    - Similar Input, Output will be 0
    - Dissimilar Input, Output will be 1
- Notation of XOR gate: A ⊕ B

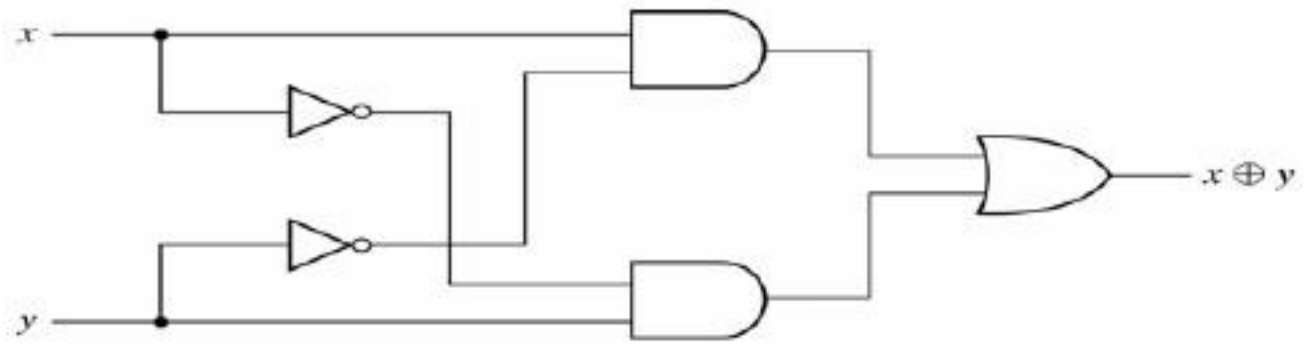
XOR Gate

- XOR Gate
  - True if either true but not both
  - Similar Input, Output will be 0
  - Dissimilar Input, Output will be 1

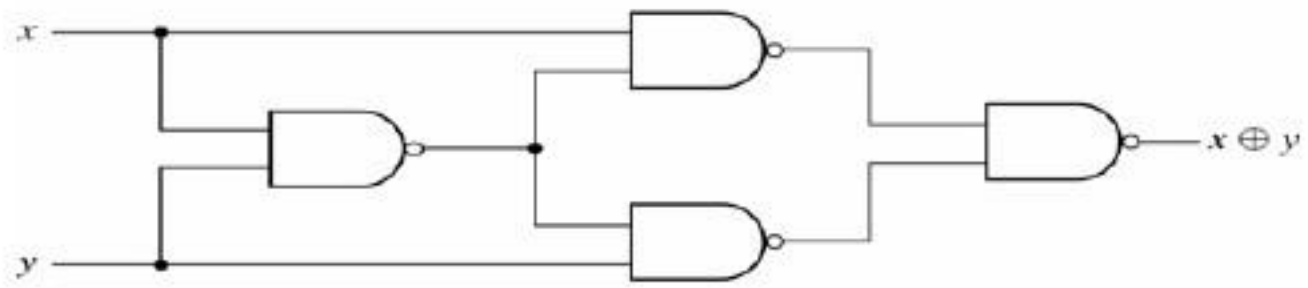
XOR Truth Table

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

XOR Gate



(a) With AND-OR-NOT gates



(b) With NAND gates

Fig. 3-32 Exclusive-OR Implementations

NAND Gate

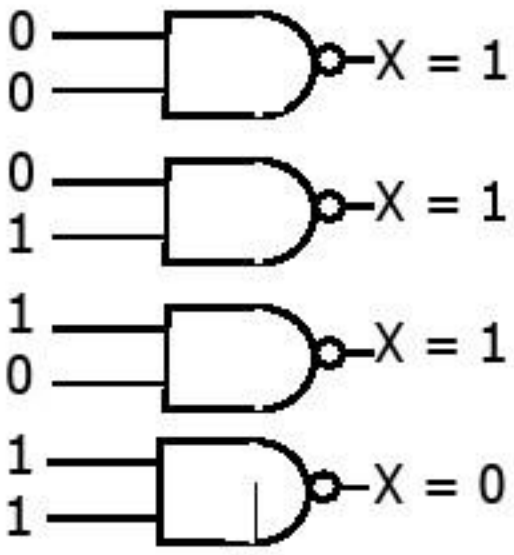
- *NAND gate*

  (NOT-AND = NAND, opposite of AND)

- *The simplest NAND gate is a two-input-one-output logic gate*
- Operation of NAND gate:
  - NAND = Output is 0 only when all inputs are 1

$\mathcal{CPF}$

NAND Gate



NAND Truth Table

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

2 0 1 4

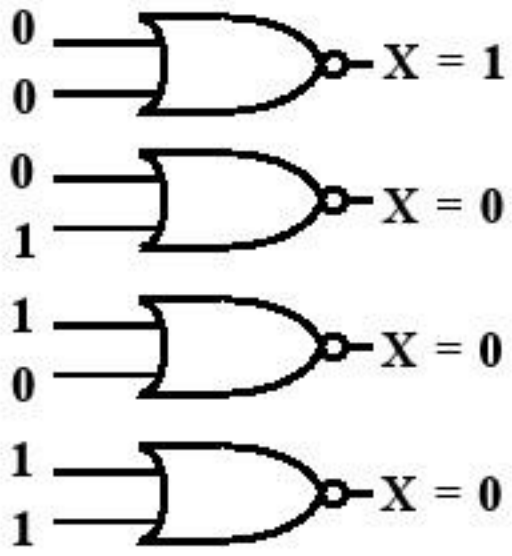
NOR Gate

- *NOR gate (NOT-OR = NOR, opposite of OR)*
- *The simplest NOR gate is a two-input-one-output logic gate*
- Operation of NOR gate:
  - NOR gate = Output is 1 only when all inputs are 0

2 0 1 4

$\mathcal{CPF}$

NOR Gate



NOR Truth Table

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

2 0 1 4

NXOR Gate

- *NXOR gate (NOT-XOR = NOR, opposite of XOR)*
- *The simplest NXOR gate is a two-input-one-output logic gate*
- Operation of XNOR gate
  - Similar Input, Output will be 1
  - Dissimilar Input, Output will be 0

NXOR Truth Table

A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

2 0 1 4

Boolean Exp → Logic Circuit

- To draw a circuit from a Boolean expression:
  - From the left, make an input line for each variable.
  - Next, put a NOT gate in for each variable that appears negated in the expression.
  - Still working from left to right, build up circuits for the sub-expressions, from simple to complex.

Logic Circuit: $A\overline{B} + \overline{(A+B)}\,B$

- Logic Circuit:

$$A\overline{B} + \overline{(A+B)}\,B$$

- Input Lines for Variables

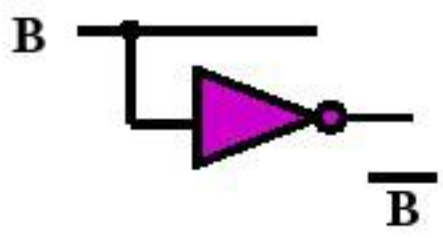
A ——

B ——

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Logic Circuit: $A\,\overline{B} + \overline{(A+B)}\,B$

- NOT Gate for B



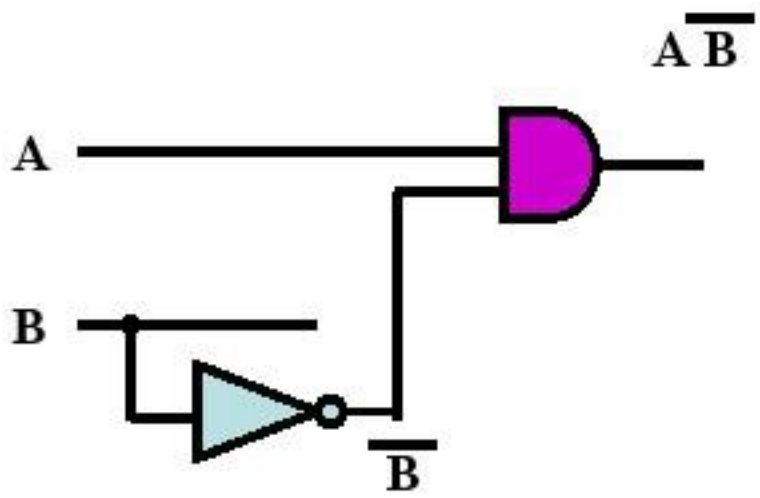
2014

71

CPF - Chapter No 4 : Number Systems and Logic Gates

Logic Circuit: $A\,\overline{B} + \overline{(A+B)}\,B$

- Sub-expression $A\,\overline{B}$



2014

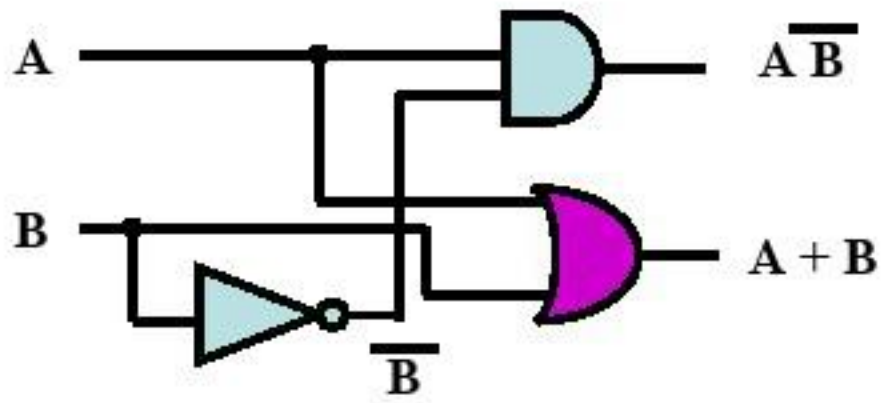
72

CPF - Chapter No 4 : Number Systems and Logic Gates

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Logic Circuit: $A\,\overline{B} + \overline{(A+B)}\,B$

- Sub-expression A + B



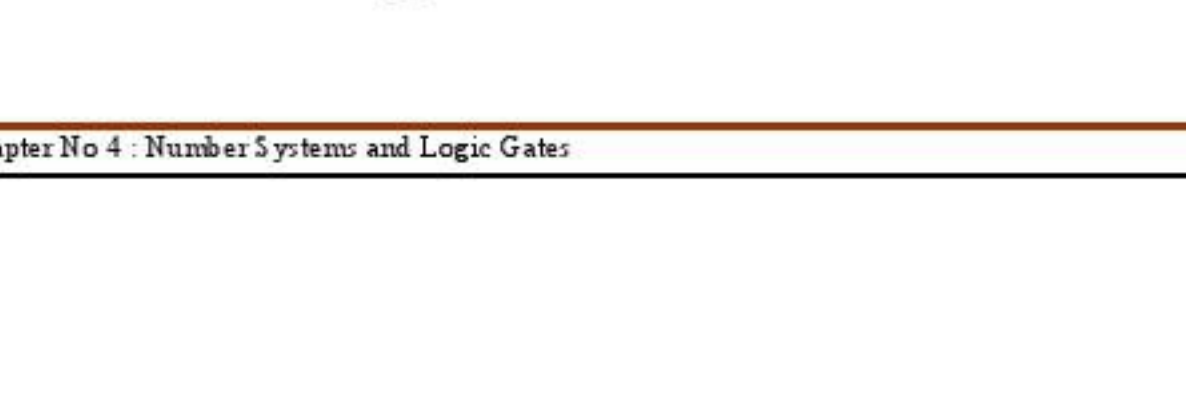
2014

73

CPF - Chapter No 4 : Number Systems and Logic Gates

Logic Circuit: $A\,\overline{B} + \overline{(A+B)}\,B$

- Sub-expression $\overline{A + B}$



2014

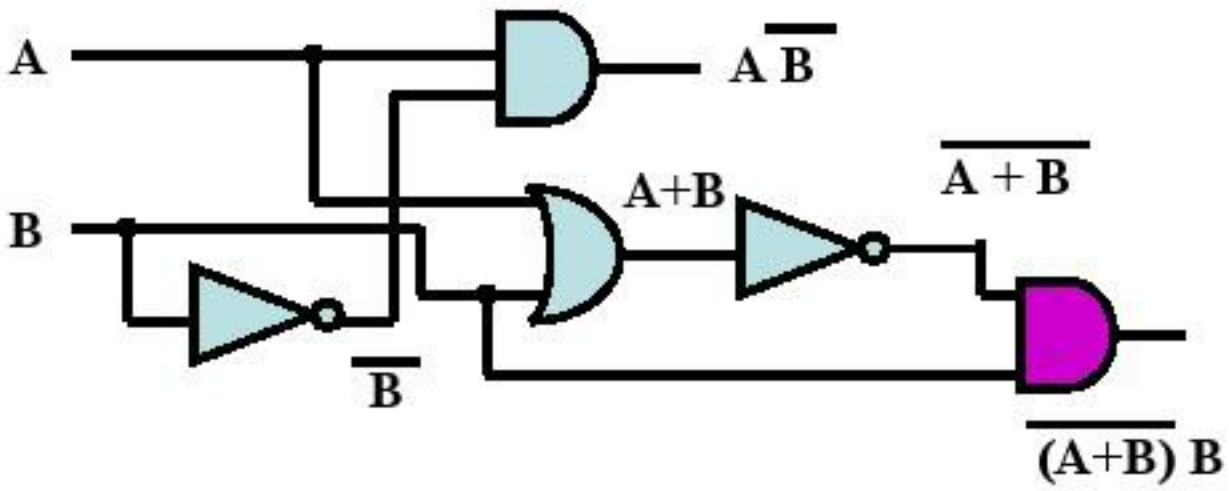
74

CPF - Chapter No 4 : Number Systems and Logic Gates

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Logic Circuit: $A\,\overline{B} + \overline{(A+B)}\,B$

- Sub-expression ($\overline{A + B}$) B



2 0 1 4

75

CPF - Chapter No 4 : Number Systems and Logic Gates

Logic Circuit: $A\,\overline{B} + \overline{(A+B)}\,B$

- Entire Expression

2 0 1 4

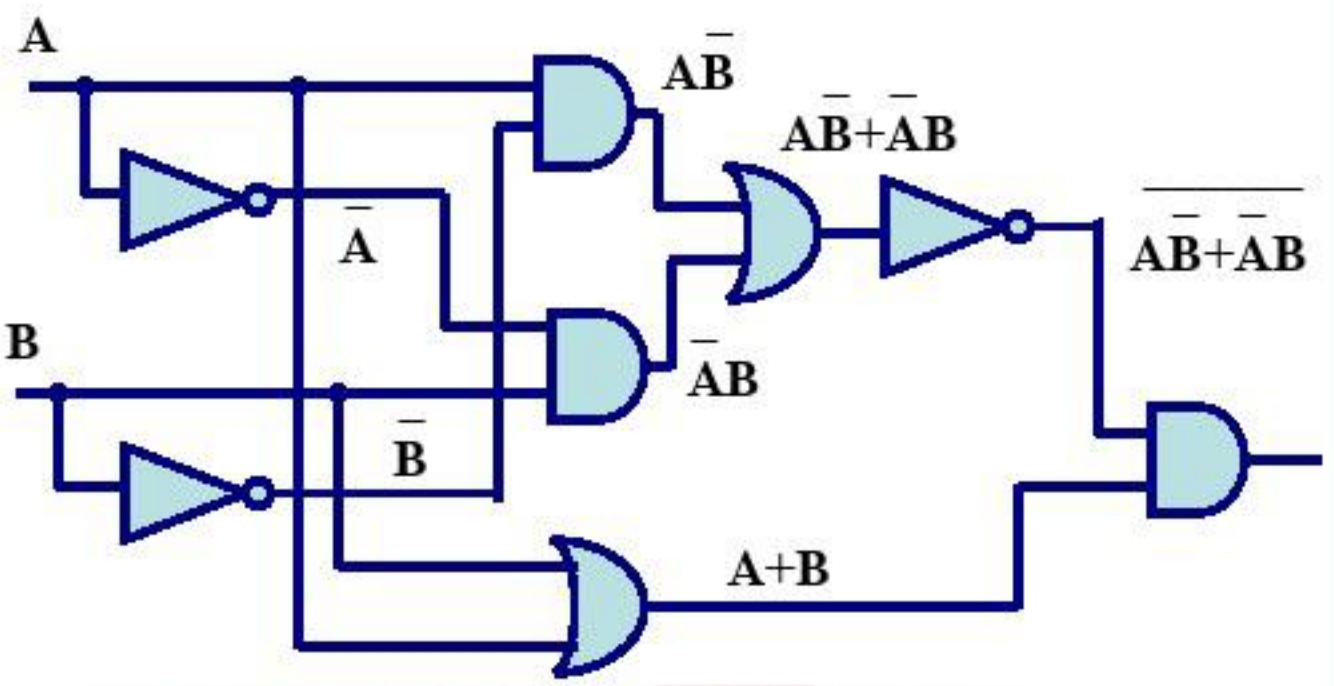
76

CPF - Chapter No 4 : Number Systems and Logic Gates

Logic Circuit → Boolean Exp

- In the opposite direction, given a logic circuit, we can write a Boolean expression for the circuit.
- First we label each input line as a variable.
- Then we move from the inputs labeling the outputs from the gates.
- As soon as the input lines to a gate are labeled, we can label the output line.
- The label on the circuit output is the result.

Logic Circuit → Boolean Exp



Simplifying Boolean Expressions

- As in ordinary algebra, some Boolean expressions can be simplified
- The logic circuit that results from a simplified expression will have fewer gates and operations
- **XOR** is *not* a Basic Function

$$A \oplus B = A\overline{B} + \overline{A}B$$

Laws of Boolean Algebra

Identity Law	Zero Law
A + 0 = A	A . 0 = 0
A . 1 = A	A + 1 = 1
Idempotent Law	**Commutative Law**
A + A = A	A + B = B + A
A . A = A	AB = BA
Associative Law	**Distributive Law**
(A+B) + C = A + (B+C)	A (B+C) = AB + AC
(AB) C = A (BC)	A + BC = (A+B) (A+C)

Laws of Boolean Algebra

Absorption Law

$A + AB = A$

$A (A+B) = A$

DeMorgan's Law

$\overline{A+B} = \bar{A}\,\bar{B}$

$\overline{AB} = \bar{A} + \bar{B}$

Complement Law

$A + \bar{A} = 1$

$A\,\bar{A} = 0$

Double Complement

$\bar{\bar{A}} = A$

Identity and Null Law

- **Identity Law**
- $A + 0 = A$
- $A \cdot 1 = A$
- $A \oplus 0 = A$

- **Null Law**
- $A + 1 = 1$
- $A \cdot 0 = 0$
- $A \oplus 1 = \bar{A}$

$\mathcal{CPF}$

Idempotence and Inverse Law

- **Idempotence Law**
- $A + A = A$
- $A \cdot A = A$
- $A \oplus A = 0$

- **Inverse Law**
- $A + \bar{A} = 1$
- $A \cdot \bar{A} = 0$
- $A \oplus \bar{A} = 1$

$$\bar{\bar{A}} = A$$

2014

Commutative and Associative Law

- **Commutative Law**
- $A + B = B + A$
- $AB = BA$
- $A \oplus B = B \oplus A$

- **Associative Law**
- $A + B + C = (A+B)+C$
  $= A+(B+C)$
- $A \ B \ C = (AB)\,C$
  $= A\,(BC)$
- $A \oplus B \oplus C = (A \oplus B) \oplus C$
  $= A \oplus (B \oplus C)$

2014

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Distributive and Absorption Law

- **Distributive Law**
- $AB + AC = A\,(B + C)$
- $AB \oplus AC = A\,(B \oplus C)$

- **Absorption Law**
- $A + AB = A$
- $A\,(A+B) = A$

2014

DeMorgan's Law

$$\overline{A + B} = \overline{A}\,\overline{B}$$

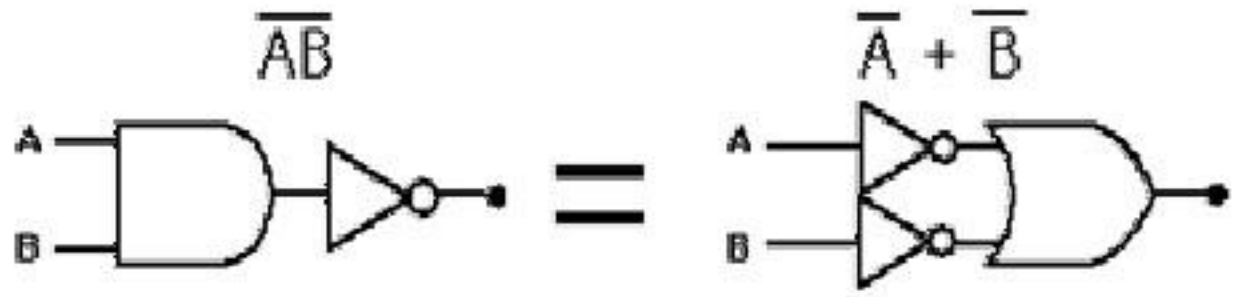
$$\overline{AB} = \overline{A} + \overline{B}$$

2014

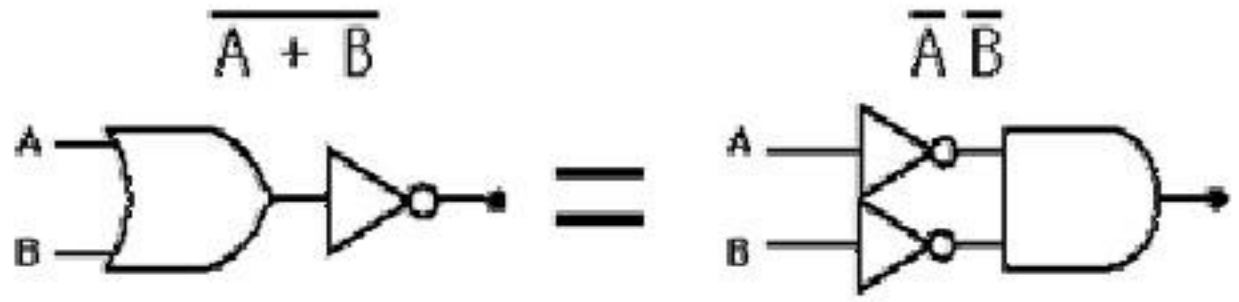
$\mathcal{CPF}$

DeMorgan's Law

- Implementation of DeMorgan's Theorem with basic gates.



A NAND gate is equivalent to an inversion followed by an OR



A NOR gate is equivalent to an inversion followed by an AND

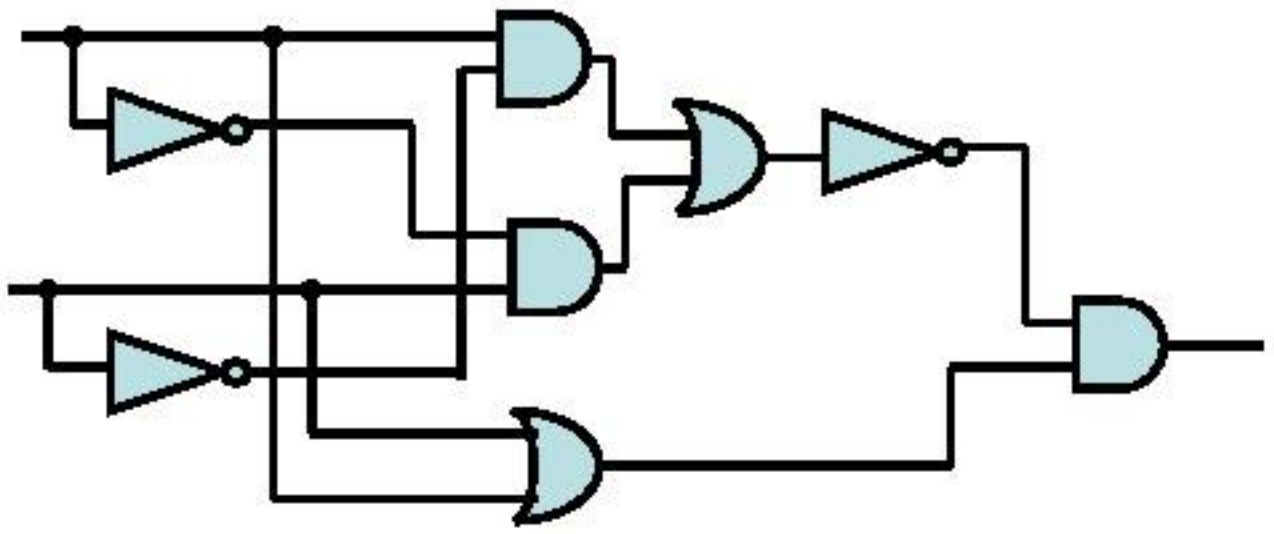
Simplification Revisited

- Once we have the BE for the circuit, perhaps we can simplify.

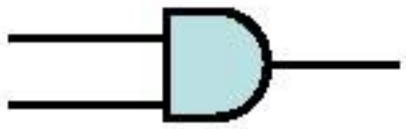
$$\begin{aligned}\left(\overline{A\,\overline{B} + \overline{A}B}\right)(A + B) &= \overline{A\,\overline{B}}\;\;\overline{\overline{A}B}\,(A + B)\\ &= \left(\overline{A} + B\right)\left(A + \overline{B}\right)(A + B)\\ &= \left(\overline{A} + B\right)\left(A + \overline{B}B\right)\\ &= \left(\overline{A} + B\right)A\\ &= \overline{A}A + BA\\ &= AB\end{aligned}$$

$\mathcal{CPF}$

Logic Circuit → Boolean Exp



Reduces to :



Example 1

$A\left(\overline{A}+B\right)$

Apply Distributive Law

$A\overline{A}+AB$

Apply Inverse Law

$0+AB$

Apply Identity Law

AB

Example 2

$AB + BC + AD + CD$

Apply Associative Law

$(AB + BC) + (AD + CD)$

Apply Distributive Law

$B(A + C) + D(A + C)$

Apply Distributive Law again

$(B + D)(A + C)$

Example 3

$A\overline{B} + (A \oplus B)$

Use Substituti on

$A\overline{B} + \left(A\overline{B} + \overline{A}B\right)$

Apply Associativ e Law

$\left(A\overline{B} + A\overline{B}\right) + \overline{A}B$

Apply Idempotenc e

$A\overline{B} + \overline{A}B$

Use Substituti on

$A \oplus B$

$C\,P\,F$

Example 4

$AB + \overline{A} + \overline{B}$

Apply Associativ e Law

$AB + \left(\overline{A} + \overline{B}\right)$

Apply DeMorgan' s Law

$AB + \overline{AB}$

Apply Inverse Law

1

Example 5

$\overline{A \oplus B}$

Rewrite using And & Or

$\overline{A\overline{B} + \overline{A}B}$

Apply DeMorgan' s Law

$\left(\overline{A\overline{B}}\right)\left(\overline{\overline{A}B}\right)$

Apply DeMorgan' s Law again

$\left(\overline{A} + \overline{\overline{B}}\right)\left(\overline{\overline{A}} + \overline{B}\right)$

$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Example 5 (continued)

$$\left(\overline{A} + \overline{\overline{B}}\right)\left(\overline{\overline{A}} + \overline{B}\right)$$

Apply Inverse Law

$$\left(\overline{A} + B\right)\left(A + \overline{B}\right)$$

Apply Distributive Law

$$\overline{A}A + \overline{A}\,\overline{B} + AB + B\,\overline{B}$$

Apply Inverse Law

$$0 + \overline{A}\,\overline{B} + AB + 0$$

Apply Identity Law

$$\overline{A}\,\overline{B} + AB$$

2 0 1 4

Example 6

$$AB+\overline{AB}$$

$$\left(\overline{AB+\overline{\overline{AB}}}\right)$$ Apply Inverse Law

$$\left(\overline{AB}\left(\overline{\overline{AB}}\right)\right)$$ DeMorgan's Law

$$\left(\overline{\overline{A}+\overline{B}}\right)\overline{(A+B)}$$ DeMorgan's Law

$$\overline{\overline{A}A+A\overline{B}+\overline{A}B+\overline{B}B}$$ Distributive Law

$$\overline{0+A\overline{B}+\overline{A}B+0}$$ Inverse Law

$$\overline{A\overline{B}+\overline{A}B}$$ Identity Law

$$\overline{A\oplus B}$$ Equivalene

2 0 1 4

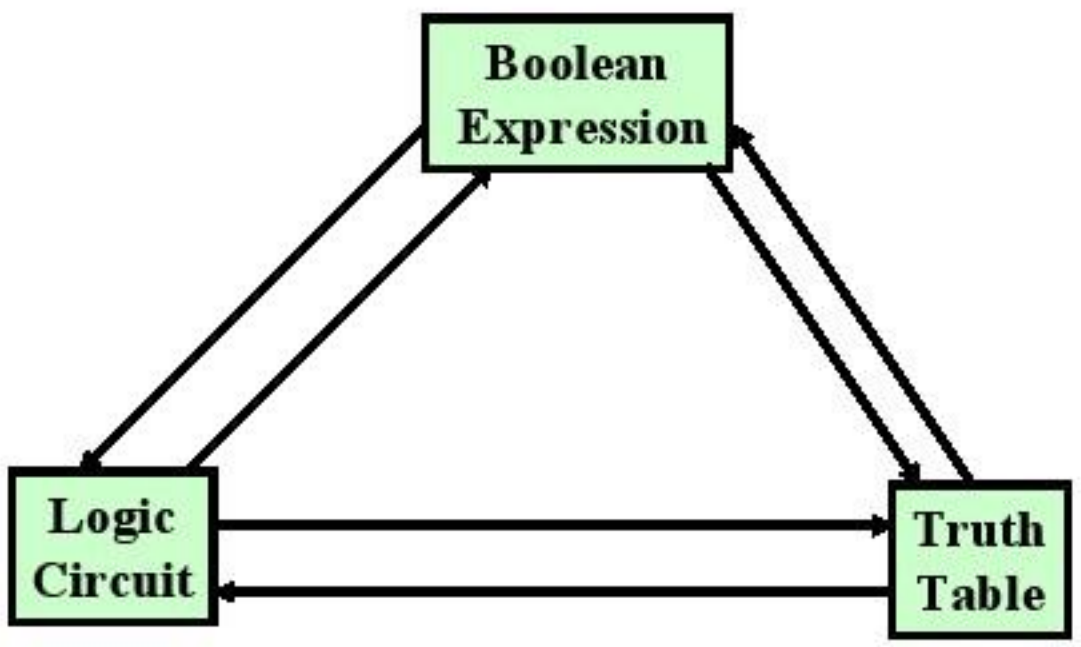
$\mathcal{C}\,\mathcal{P}\,\mathcal{F}$

Algebraic Manipulation Examples

1. $X + XY = X(1+Y) = X$
2. $XY + XY' = X(Y + Y') = X$
3. $X + X'Y = (X + X')(X + Y) = X + Y$
4. $X(X + Y) = X + XY = X(1 + Y) = X$
5. $(X + Y)(X + Y') = X + XY' + XY + YY'$
   $= X(1 + Y') + XY$
   $= X \cdot 1 + XY$
   $= X + XY$
   $= X(1 + Y)$
   $= X$
6. $X(X' + Y) = XX' + XY = XY$

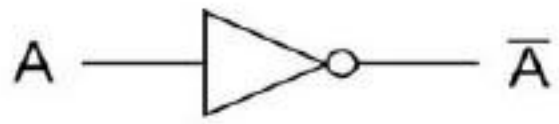
CPF - Chapter No 4 : Number Systems and Logic Gates 97

The Boolean Triangle

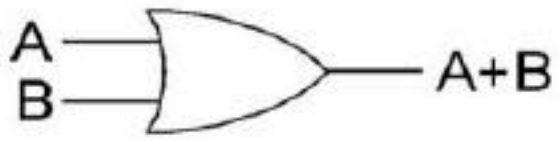


CPF - Chapter No 4 : Number Systems and Logic Gates 98

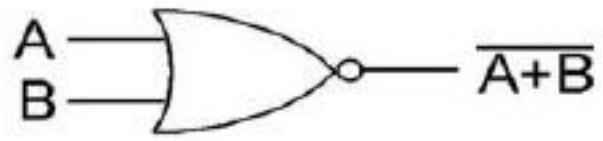
Review Basic Logic Gates



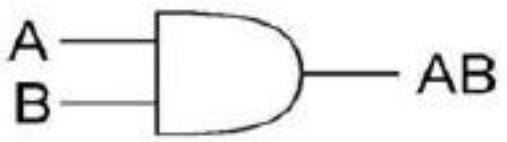
NOT



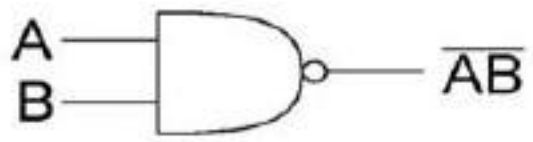
OR



NOR



AND



NAND

2 0 1 4

99

CPF - Chapter No 4 : Number Systems and Logic Gates

Conclusion

- Logic Gates and Boolean Algebra are the bridge between symbolic logic and electronic digital computing.

2 0 1 4

100

CPF - Chapter No 4 : Number Systems and Logic Gates